

ERBOT: A Unified Entity Resolution Pipeline

Vignette –

Xinmin Chu

```
knitr::opts_chunk$set(  
  collapse = TRUE,  
  comment = "#>",  
  fig.width = 7,  
  fig.height = 5  
)
```

1. Introduction

Entity resolution (ER) — also called deduplication, record linkage, or entity matching — answers a deceptively simple question: which records in my data refer to the same real-world entity? Customer databases contain "Acme Corp" and "ACME Corporation"; bibliographic databases contain the same paper cited with slightly different author lists; product catalogs contain the same item under different SKUs. ERBOT is an R package that solves this problem end-to-end through a nine-stage pipeline, from raw table to cluster labels, with honest evaluation built in.

Design principles

1. "Data in, entity out." One call — `er_run()` — takes any tabular dataset and returns cluster labels. Every stage is also usable standalone for researchers who want to swap components.
2. NA-aware everything. Real data have missing values. Similarities, weights, and combination all handle NA explicitly instead of silently dropping or imputing it.
3. Honest evaluation by default. Ground truth, when supplied, is used only for evaluation — never for model selection (choice of k , choice of clustering method) — unless the user explicitly opts into `supervised_selection = TRUE`. This discipline is enforced in code, not just documented.
4. Method coverage over method advocacy. Eight unsupervised clustering methods plus 11 supervised classifiers, five weight-learning strategies, four blocking methods, and an ablation framework that compares them fairly. The package helps you find which method works for your data, not which method the authors prefer.

2. Installation

```
# Core dependencies  
install.packages(c(  
  "data.table", "dplyr", "tibble", "stringr", "stringi", "tidyr",  
  "purrr", "readr", "readxl", "text2vec", "Matrix", "irlba",  
  "stringdist", "igraph", "FNN", "cluster", "scales",  
  "grid", "gridExtra", "magrittr", "jsonlite"  
)  
)  
  
# Optional but recommended  
install.packages("aricode") # ARI and related metrics  
remotes::install_github("ddegras/GCMER") # agreement metrics & graph coloring  
  
# Install erbot
```

```
remotes::install_github("xinminchu/erbot", upgrade = "never")
```

R $\geq$ 4.1 is required.

3. Quick start

```
library(erbot)

# Any data.frame, file path, or benchmark keyword ("cora", "affiliation", "d10k")
res <- er_run(
  data    = my_data,
  truth   = my_truth,      # optional: named integer vector, or data.frame(id, cluster_id)
  verbose = TRUE
)

res$clusters      # integer vector of final cluster labels (length n)
res$performance   # tibble of ARI, B-cubed F, homogeneity, completeness, ...
res$predictions   # tibble(id, cluster_id), ready for write-out
print(res)        # compact summary
```

With `out_dir` set, `er_run()` also writes `er_predictions.csv` and `er_performance.csv` to disk.

4. The nine-stage pipeline

Every `er_run()` call executes the same nine stages. Understanding them is understanding the package.

Stage 1 — Load (`erload()`)

Ingests the input, which may be a `data.frame`, a file path (CSV/TSV/Excel), or a benchmark keyword ("cora", "affiliation", "d10k"). Column names are normalized to lowercase; the result is a tibble ready for diagnosis.

Stage 2 — Diagnose (`erdiagnose()`)

Profiles every field and decides how the pipeline should treat it:

- ID column auto-detection (candidates: `id`, `record_id`, `paper_id`, ...).
- Mode detection: dedup (one table) vs link (two sources, detected

via a `source_id` column).

- Field typing: for each column, the missing-value rate, cardinality, and a recommended similarity method (jw for short strings, cosine for long text, numeric for numbers, exact for codes).

- Blocking recommendation: whether blocking is needed at all (`blocking_needed`), and which method/key to use.

The diagnosis object drives all downstream "auto" defaults, so a user who changes nothing still gets a sensible configuration.

Stage 3 — Block (`erblock()`)

Naive pairwise comparison is $O(n^2)$. Blocking generates a candidate pair set that is orders of magnitude smaller while (hopefully) retaining all true matches. Four methods:

Method	Idea
--------	------

`"none"`	All pairs (only for small n; guarded by `max_p
`"standard"`	Exact match on a blocking key
`"prefix"`	Shared prefix of length `prefix_len` (default
`"sn"`	Sorted neighbourhood with window `sn_window` (

`er_block_stats()` reports the reduction ratio (fraction of pairs pruned) and, when truth is available, pair completeness (fraction of true matches retained). A block that prunes 99% of pairs but loses 30% of true matches is a bad block — this function makes that visible.

A built-in guard estimates $C(m, 2)$ before allocating: if the candidate count would exceed `max_pairs` (default 2M), blocking refuses to produce an unbounded pair set instead of crashing with OOM.

Stage 4 — Similarity (`ersimilarity()`)

Computes per-field similarities only on candidate pairs, stored as a named list of numeric vectors (`sim_list`). Key properties:

- NA-aware: if either value is missing, the field contributes no signal rather than a spurious 0.

- Per-field methods: Jaro-Winkler (`jw`) for names, TF-IDF cosine for long text, normalized numeric distance, exact match for codes. The spec can come from `er_diagnose()` or be supplied by hand.

Stage 5 — Weights (`erweights()`)

Not all fields are equally informative. Five strategies:

Method	Weight α	Needs truth?
`"equal"`	1/K	No
`"fellegi_sunter"`	$\log(m_k / u_k)$ via EM	No
`"ari"`	per-field ARI vs truth	**Yes**
`"bimodal"`	bimodality of the similarity d	No
`"variance"`	variance of the similarity dis	No

"auto" picks "ari" when truth is supplied, "fellegi_sunter" otherwise. The ARI method supports nested inner CV (`inner_k`) so that using ARI both for weight learning and for final evaluation does not leak.

Stage 6 — Combine (`ercombine()`)

NA-aware weighted combination of the per-field similarities into a single score per pair, then `er_pairs_to_sparse()` builds the symmetric sparse similarity matrix S ($n \times n$, `dgCMatrix`). Everything downstream works from S .

Stage 7 — Cluster (`erclusterall()`)

Eight unsupervised clustering methods run on S , plus 11 supervised classifiers when truth is available:

- Centroidal (need k^*): `hclust_avg`, `hclust_ward`, `pam`. Features are derived from S via SVD (exact dense SVD for small n , `irlba` otherwise); k is auto-tuned over `k_grid` by silhouette when not given.

- Graph-based: `louvain`, `leiden`, `threshold_cc` (connected components on pairs above threshold), `label_prop` (label propagation), `gc` (graph coloring).
- Supervised (need truth): the 11 classifiers from `er_supervised_classifiers()` — logistic (Firth penalized by default, so separation doesn't diverge), LDA, QDA, KNN, weighted KNN, tree, random forest, `xgboost`, `nnet`, Fellegi-Sunter, radial SVM — trained on pair features and used to classify pairs before clustering.

A sparsity guard (`.check_dense_n()`, default `n = 5000`) prevents accidental dense-matrix blowups in HC/PAM/GC on large data.

Stage 8 — Merge (`ermerge()`)

Post-processing of the clustering ensemble:

- "consensus" (default): keeps pairs on which a fraction (`consensus_alpha`) of methods agree — unsupervised.
- "transitivity": connected components on pairs with $S \geq \text{threshold}$.
- "best": picks the method with highest ARI vs truth — supervised, warns loudly, and only sees truth under `supervised_selection = TRUE`.
- "none": skip.

Stage 9 — Evaluate (`erevaluate()`)

When truth is supplied, a 14-column metric table per method:

- External: ARI, AMI, B-cubed precision/recall/F, pairwise precision/recall/F, homogeneity, completeness, V-measure.
- Label alignment is done by record name, never by position, with a length check against the ID vector enforced.

Without truth, `er_unsupervised_quality()` reports within/between similarity, the similarity gap, and silhouette.

Stage 10 — Output

`er_run()` returns an `er_result` object (with a `print` method) containing clusters, all per-method clusters, the performance table, the diagnosis, blocking stats, weights, and parameters. With `out_dir`, predictions and performance are written to CSV.

5. Honesty discipline: how the package prevents fooling yourself

Getting evaluation right is where most ER toolkits stumble. This section describes the discipline built into the package.

1. Truth never drives model selection by default. `supervised_selection = FALSE` is the default. Truth is used for evaluation (stage 9) and optionally for weight learning (weights = "ari"), but never for choosing `k` or picking the best method. `merge = "best"` without the flag warns and is blocked from seeing truth.
2. Entity-level splits, not record-level splits. `er_split()`, `er_cv()`, and `er_protocol_b()` split by entity (all records of an entity go to the same fold). Record-level splits leak near-duplicates across train/test and

inflate scores. 3. True held-out evaluation. `er_protocol_b()` partitions entities into Selection and Test sets; `k` and `method` are chosen on Selection (with inner entity-level CV); the Test set is clustered and evaluated exactly once. 4. Seeded reproducibility. All stochastic steps take seeds; RNG state is saved and restored (`.rng_save()`) so a pipeline run doesn't perturb the caller's random stream.

6. Supporting frameworks

Beyond the pipeline, the package ships research infrastructure:

- Ablation (`er_ablation_all()`, `er_ablation_table()`): full factorial over weight conditions (uniform / Fellegi-Sunter EM / full-data ARI / CV-learned ARI) × blocking conditions (none / blocked), with `print_ablation()` summaries.
- Tuning (`er_tune()`): honest three-way train/validation/test protocol for hyperparameter search; internal vs external objectives.
- CV protocols (`er_protocol_a()`, `er_protocol_b()`): the "honest" evaluation protocols used in the paper.
- Supervised pair classification (`er_pair_features()`, `er_pair_classify()`, `er_supervised_classifiers()`): 11 classifiers with a validity gate — structurally unsuitable configurations return `valid = FALSE` with a reason instead of silently producing garbage.
- Baselines (`er_baseline_splink()`, `er_baseline_deepmatcher()`, `er_baseline_ditto()`): wrappers that call out to Python for probabilistic (Splink) and deep-learning (DeepMatcher, Ditto) baselines.
- Scenarios (`er_list_scenarios()`, `er_run_scenario()`): 21 preset configurations with friendly error messages, built for teaching and demos.
- Reporting (`er_save_report_pdf()`, `er_draw_table()`): PDF reports with performance tables and diagnostic plots.

7. Worked example

A tiny self-contained deduplication task: 12 records, 4 true entities (each with small name variations).

```
library(erbot)

toy <- data.frame(
  id   = paste0("r", 1:12),
  name = c("Acme Corporation", "Acme Corp", "ACME Corp.",
           "Beacon Technologies", "Beacon Tech", "Beacon Technology Inc",
           "Crestline Labs", "Crestline Lab", "Crestline Laboratories",
           "Driftwood Systems", "Driftwood Sys", "Driftwood System Inc"),
  stringsAsFactors = FALSE
)
truth <- setNames(rep(1:4, each = 3), toy$id)

res <- er_run(
  data      = toy,
```

```

truth      = truth,
cluster_methods = c("louvain", "threshold_cc", "hclust_avg"),
verbose     = FALSE
)

print(res)
#> er_result: n=12 records, 4 final clusters
#> Performance:
#>   method      ARI Bcubed_F Vmeasure
#>   louvain     1.000   1.000   1.000
#>   threshold_cc 0.000   0.400   0.000
#>   hclust_avg   0.891   0.941   0.946
#>   final       1.000   1.000   1.000

# Inspect what the pipeline decided at each stage
res$diag$recommended_block_method # which blocking was chosen
res$weights                       # learned field weights
res$blocking$stats$reduction_ratio # fraction of pairs pruned
res$predictions                  # id -> cluster_id

```

Note how the three methods disagree (threshold_cc collapses here at the default threshold, hclust_avg is close but not perfect) yet the consensus merge (final) recovers the correct 4 clusters — this is exactly why the package runs an ensemble instead of betting on one method.

Stage-by-stage manual control is equally possible:

```

df <- er_load(toy)
diag <- er_diagnose(df)
pairs <- er_block(df, diag = diag, method = "prefix", prefix_len = 3)
sim <- er_similarity(df, pairs, diag = diag)
wt <- er_weights(sim, method = "fellegi_sunter")
S <- er_pairs_to_sparse(pairs, er_combine(sim, weights = wt), nrow(df))
labs <- er_cluster(S, method = "louvain")
er_evaluate(list(louvain = labs), truth = truth, id_vec = df$id)

```

8. Function reference map

Task	Entry points
Full pipeline	<code>`er_run()`</code>
Load / diagnose	<code>`er_load()`</code> , <code>`er_diagnose()`</code> , <code>`er_data_profile`</code>
Blocking	<code>`er_block()`</code> , <code>`er_block_stats()`</code>
Similarity	<code>`er_similarity()`</code> , <code>`er_pairwise_stringdist()`</code> ,
Weights	<code>`er_weights()`</code> , <code>`er_combine()`</code> , <code>`er_pairs_to_s`</code>
Clustering	<code>`er_cluster()`</code> , <code>`er_cluster_all()`</code> , <code>`er_merge(</code>
Evaluation	<code>`er_evaluate()`</code> , <code>`er_bcubed()`</code> , <code>`er_vmeasure()</code>
Honest protocols	<code>`er_protocol_a()`</code> , <code>`er_protocol_b()`</code> , <code>`er_cv()</code>
Ablation	<code>`er_ablation_all()`</code> , <code>`er_ablation_table()`</code> , <code>`p`</code>
Supervised	<code>`er_pair_features()`</code> , <code>`er_pair_classify()`</code> , <code>`e`</code>
Baselines	<code>`er_baseline_splink()`</code> , <code>`er_baseline_deepmatch`</code>
Scenarios/demos	<code>`er_list_scenarios()`</code> , <code>`er_run_scenario()`</code> , <code>`e`</code>
Reporting	<code>`er_save_report_pdf()`</code> , <code>`er_draw_table()`</code> , <code>`pr`</code>
Datasets	<code>`er_build_restaurant()`</code> , <code>`er_build_dblp_acm()`</code>

```
sessionInfo()
```